

ICS小班课回课-指令集并行优化技术

王嘉怡 2400013107

2025.11.5

CPE

(1) 基于课件第34~36页，解读combine1要评测的操作，然后解释CPE的定义。

```
1  #define IDENT 0 // 或者1
2  #define OP + // 或者-*
3  void combine1(vec_ptr v, data_t* dest) {
4      long int i;
5      *dest = IDENT;
6      for (i = 0; i < vec_length(v); i++) {
7          data_t val;
8          get_vec_element(v, i, &val);
9          *dest = *dest OP val;
10     }
11 }
```

combine1: “合并运算”，计算 IDENT 和 v 中所有元素依次进行 OP 操作的结果。

CPE = cycles per OP: 一条指令执行了多少个时钟周期

程序的一步步优化

(2) 基于课件第37~39页，并结合教材，逐个解读combine1/2/3/4的优化操作和效果。

代码移动

combine2: 将循环测试条件中不断调用的 vec_length(v)，赋值给局部变量 length。

```
1  /* Move call to vec_length out of loop */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v); // improvement
6      *dest = IDENT;
7      for (i = 0; i < length; i++) { // improvement
8          data_t val;
```

```

9         get_vec_element(v, i, &val);
10        *dest = *dest OP val;
11    }
12 }

```

- **代码移动**：识别执行多次但结果不会改变的运算。

局限性：编译器可能无法正确识别**函数的副作用**，因而非常保守，从而通常需要程序员显式完成代码移动。

减少过程调用

combine3：（注意到这个代码中 `get_vec_element` 不会造成溢出）把循环内部调用的 `get_vec_element(v)` 函数改为循环外调用 `get_vec_start(v)`。

```

1  data_t *get_vec_start(vec_ptr v) {
2      return v->data;
3  }
4
5  /* Direct access to vector data */
6  void combine3(vec_ptr v, data_t *dest)
7  {
8      long i;
9      long length = vec_length(v);
10     data_t *data = get_vec_start(v); // improvement
11     *dest = IDENT;
12     for (i = 0; i < length; i++) {
13         *dest = *dest OP data[i];
14     }
15 }

```

经过测试，这一改进没有造成性能的提升，这是因为其它操作行程瓶颈，暂时限制了性能的上限。

消除不必要的内存引用

combine4：以前每次循环时都会进行 `*dest` 的读和写，通过临时变量 `acc`，减少内存读写次数。这一修改显著提高了代码效率。

```

1  /* Accumulate result in local variable */
2  void combine4(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7      data_t acc = IDENT; // improvement
8

```

```

9      for (i = 0; i < length; i++) {
10         acc = acc OP data[i]; // improvement
11     }
12     *dest = acc; //improvement
13 }

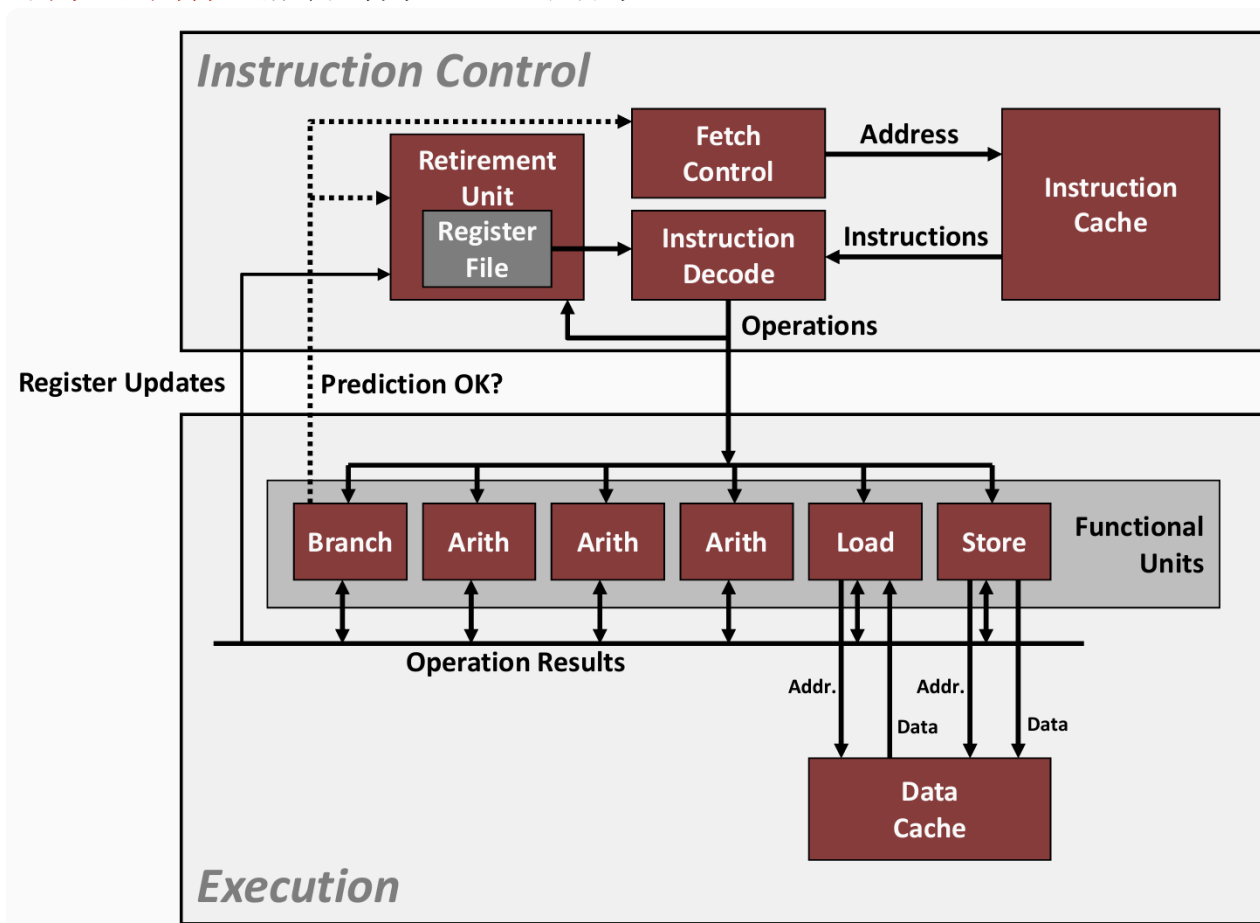
```

现代超标量流水线处理器

(3) 基于课件第40页的现代超标量流水线处理器设计结构图，对照此前课程学过的五级单流水线处理器，逐个流水级解读二者的异同。

- **特点：**

- **超标量superscalar**：每个周期执行多个操作
- **乱序out-of-order**：指令执行顺序不一定与机器级程序中的顺序一致
- **两个主要部分**：指令控制单元ICU&执行单元EU



阶段	现代处理器	流水线处理器
取指Fetch	ICU从 指令高速缓存 中读取指令；取指发生在执行指令很早之前。 分支预测技术 、 投机执行技术 。	从PC读取指令；取指后直接执行。分支预测较简单。
译码Decode	将指令分解为一组 微指令 。	

阶段	现代处理器	流水线处理器
执行 Execute	EU在一个周期接收多个操作，分派到一组 功能单元 中。浮点运算。	一个周期一条，ALU负责全部功能工作。
访存 Memory	专门的 load 单元	
写回Write Back	专门的 store 单元	
其它	退役单元 ：只有在指令退役时才会更新程序寄存器	

功能单元

(4) 基于课件第43页，配合第40页的结构图（不是一一对应，但可以参照），解读CPU中功能部件数量和性能指标，然后结合第42页讲解功能部件的流水化和非流水化的区别。

功能单元	数量
整数操作+浮点运算	2
整数运算	2
加载+地址计算	2
存储+地址计算	1
存储	1

性能指标	延迟	发射时间	容量
整数加法	1	1	4
整数乘法	3	1	1
整数除法	3~30	3~30	1
浮点加法	3	1	1
浮点乘法	5	1	2
浮点除法	3~15	3~15	1

功能部件的流水线：每个阶段完成一部分运算，从而可以在每个时钟周期开始一个新的运算（即使运算需要多个周期）。

循环展开

2 × 1循环展开

(5) 基于课件第44~47页，分析“2x1”版的优化方式和效果。

优化方式：一次循环处理数组的n个元素，最后处理可能的余数。

```
1  for (i = 0; i < length; i++) {
2      acc = acc OP data[i] OP data[i+1];
3  }
4  for (; i < length; i++) {
5      acc = acc OP d[i];
6  }
```

优化效果：减少了循环测试条件的次数，从而降低了整体开销。

2 × 1a循环展开

(6) 基于课件第48~50页，分析“2x1a”版的优化方式和效果。

优化方式：重新结合运算（加括号）

```
1  for (i = 0; i < length; i++) {
2      acc = acc OP (data[i] OP data[i+1]);
3  }
4  for (; i < length; i++) {
5      acc = acc OP d[i];
6  }
```

优化效果：减少计算中关键路径上操作的数量，**提高并行性**。

注意：要保证运算是可结合的（浮点运算）。效果不一定好。

2 × 2循环展开

(7) 基于课件第51~53页，分析“2x2”版的优化方式和效果。

优化方式：将一组合并运算分为多个部分（多个变量），最后合并结果

```
1  data_t acc1 = IDENT;
2  data_t acc2 = IDENT;
3  for (i = 0; i < length; i++) {
4      acc1 = acc1 OP data[i];
5      acc2 = acc2 OP data[i+1];
6  }
7  for (; i < length; i++) {
8      acc1 = acc1 OP d[i];
9  }
10 *dest = acc1 OP acc2; // improvement
```

优化效果：减少数据相关，**提高并行性**

评测与局限性

(8) 基于课件第54~57页，解读该评测方法和评测结果。

评测维度：L、K： $L \times K$ 的循环展开（注意L必须是K的倍数）

评测标准：参考机上运行的CPE

- **延迟界限：**完成一条指令需要的最短时钟周期→数据依赖的上限
- **吞吐量界限：**一个时钟周期内最多可以完成几个指令→并行的上限

评测结果：大体循环展开的K和L越大，CPE越低

局限性：

1. K的局限：寄存器有限，并行的变量数K过大超过寄存器数，程序会把变量入栈
2. L的局限：L过大，理论上等价于；余数处理问题
→结果： 8×8 或 10×10 次循环展开达到最佳

SIMD: Single-Instruction, Multiple-Data

优化方式：

- **向量寄存器：**`%ymm0~%ymm15`，每个长32字节
 - AVX(Advanced Vector Extension)指令**并行地**计算向量寄存器中的所有值
- 优化效果：**极大地提高了吞吐量界限

转移指令

(10) 基于课件第61~68页，简要分析转移指令带来的性能损失、优化方式和效果。

性能损失：预测错误处罚，19个时钟周期

- 来源：**投机执行技术**：确定分支之前就已经译码甚至执行
- 优化方式：**
- 机器：**分支预测技术**
 - 程序员：
 - 书写适合用**条件数据传送**实现的代码